

# USART Driver Analysis

## Overview

The USART (Universal Synchronous/Asynchronous Receiver Transmitter) driver provides an abstraction layer over the STM32F4 USART peripheral registers. It enables applications to perform serial communication without directly manipulating hardware registers.

The driver supports:

- Peripheral initialization and configuration
- Baud rate generation
- Polling-based transmission and reception
- Interrupt-driven transmission and reception
- Hardware flow control (CTS/RTS)
- Interrupt management and priority configuration
- Error detection and event notification through callbacks

The driver hides register-level complexity and exposes simple APIs for configuring and using USART communication.

---

## Driver Architecture

The driver follows a layered approach:

### Application Layer

Uses driver APIs such as:

- USART\_Init()
- USART\_SendData()
- USART\_ReceiveData()
- USART\_SendDataIT()
- USART\_ReceiveDataIT()

## Driver Layer

Handles:

- USART register configuration
- Baud rate calculations
- Interrupt processing
- State management

## Hardware Layer

STM32 USART peripheral registers:

- CR1
  - CR2
  - CR3
  - BRR
  - SR
  - DR
- 

## Important APIs

### USART\_Init()

Initializes the USART peripheral according to user configuration.

Configures:

- TX/RX mode
- Word length
- Parity control
- Stop bits
- Hardware flow control
- Baud rate

Abstraction:

The application provides configuration through a handle structure while the driver programs all required registers internally.

## **USART\_SetBaudRate()**

Calculates and configures the Baud Rate Register (BRR).

Formula:

$$\text{USARTDIV} = \text{Peripheral Clock} / (\text{Oversampling} \times \text{Baud Rate})$$

Features:

- Supports oversampling by 8 and 16
- Uses fixed-point arithmetic
- Automatically computes mantissa and fraction values

Abstraction:

The user only specifies the baud rate. Register calculations are handled internally.

---

## **USART\_PeripheralControl()**

Enables or disables the USART peripheral.

Internally controls:

- UE (USART Enable) bit in CR1

Abstraction:

The application does not need direct register access.

---

## **USART\_PeriClockControl()**

Enables or disables USART peripheral clocks.

Internally modifies RCC clock control registers.

Abstraction:

Hides RCC register configuration from the application.

## **USART\_SendData()**

Polling-based data transmission.

Operation:

1. Wait for TXE flag.
2. Load data into DR register.
3. Repeat until all bytes are transmitted.
4. Wait for TC flag before completion.

Supports:

- 8-bit communication
- 9-bit communication
- Parity enabled/disabled modes

Abstraction:

The application sends a buffer while the driver manages status flags and register access.

---

## **USART\_ReceiveData()**

Polling-based data reception.

Operation:

1. Wait for RXNE flag.
2. Read received data from DR.
3. Store data into user buffer.
4. Repeat until requested bytes are received.

Supports:

- 8-bit frames
- 9-bit frames
- Parity handling

Abstraction:

The application receives data through a simple API without managing status flags.

## **USART\_SendDataIT()**

Starts interrupt-driven transmission.

Operation:

- Stores buffer information in handle structure
- Sets transmission state to busy
- Enables TXE and TC interrupts

Actual transmission is performed inside the interrupt handler.

Abstraction:

Provides non-blocking transmission.

---

## **USART\_ReceiveDataIT()**

Starts interrupt-driven reception.

Operation:

- Stores receive buffer information
- Sets reception state to busy
- Enables RXNE interrupt

Reception continues automatically through interrupts.

Abstraction:

Provides non-blocking reception.

---

## **USART\_GetFlagStatus()**

Reads USART status flags.

Examples:

- TXE
- RXNE
- TC
- CTS

Abstraction:

Returns flag status without exposing register details.

---

## **USART\_ClearFlag()**

Clears supported status flags.

Used for:

- CTS flag
- TC flag
- LIN Break Detection flag

Abstraction:

Provides a simple API for flag management.

---

## **USART\_IRQInterruptConfig()**

Configures NVIC interrupt enable or disable.

Functions:

- Enables IRQ through ISER registers
- Disables IRQ through ICER registers

Abstraction:

Hides NVIC register access from the application.

---

## **USART\_IRQPriorityConfig()**

Configures interrupt priority.

Operation:

- Calculates correct NVIC priority register
- Programs implemented priority bits

# Interrupt Handling

## USART\_IRQHandling()

Central interrupt service routine for USART events.

Handles:

### Transmission Events

- TXE (Transmit Data Register Empty)
- TC (Transmission Complete)

### Reception Events

- RXNE (Receive Data Register Not Empty)

### Flow Control Events

- CTS

### Communication Events

- IDLE detection

### Error Events

- Overrun Error (ORE)
- Noise Error (NE)
- Framing Error (FE)

The handler updates driver states, transfers data, clears flags, and generates application callbacks.

Abstraction:

The application receives high-level event notifications without handling hardware interrupt logic.

## Error Handling

The driver supports detection of:

### Overrun Error (ORE)

Occurs when new data arrives before previously received data is read.

Cleared using:

USART\_ClearOREFlag()

---

### Noise Error (NE)

Occurs when excessive noise corrupts received data.

---

### Framing Error (FE)

Occurs when stop-bit detection fails.

---

## Callback Mechanism

### USART\_ApplicationEventCallback()

Weak callback function used for notifying the application layer.

Events include:

- USART\_EVENT\_TX\_CMPLT
- USART\_EVENT\_RX\_CMPLT
- USART\_EVENT\_IDLE
- USART\_EVENT\_CTS
- USART\_ERR\_ORE
- USART\_ERR\_FE
- USART\_ERR\_NE

Users can redefine this function in the application layer without modifying the driver source code.



Example use cases:

- Toggle LEDs
  - Signal task completion
  - Process received packets
  - Handle communication errors
- 

## Level of Abstraction Provided

The USART driver abstracts:

- Baud rate calculations
- Register configuration
- Status flag handling
- Interrupt configuration
- NVIC programming
- Transmission state management
- Reception state management
- Error handling
- Event notification

This allows application code to focus on communication logic while the driver manages all low-level USART hardware operations.

---

## Conclusion

The USART driver provides a complete abstraction layer for STM32 serial communication. It supports both polling and interrupt-driven operation, handles peripheral configuration, manages communication states, detects errors, and notifies the application through callbacks. This modular design improves code portability, readability, and maintainability while hiding hardware-specific implementation details.